

# Ecological Models

Agent-based ecology with algebraic rewriting

Simon Frost

2026-03-30

## 1 Overview

AlgebraicABMs provides a natural framework for ecological modeling, where populations are represented as objects in an ACSet and demographic events (birth, death, predation, migration) are rewrite rules. This vignette demonstrates three ecological models:

1. **Birth-Death-Immigration:** The simplest population model with analytical steady state
2. **Predator-Prey on a Graph:** Lotka-Volterra style dynamics with spatial structure
3. **Metapopulation Dynamics:** Migration between connected habitat patches

Each model highlights different AlgebraicABMs features, from basic vertex creation/deletion to state-dependent timers and graph-based interactions.

## 2 Setup

```
using AlgebraicABMs
using Catlab, AlgebraicRewriting
using Distributions: Exponential
using Random
Random.seed!(42)

TaskLocalRNG()
```

## 3 Example 1: Birth-Death-Immigration Process

The birth-death-immigration (BDI) process is a foundational ecological model. Organisms are born, die, and immigrate into a habitat. At equilibrium, the expected population size has a known analytical form.

### 3.1 Model Structure

Each organism is a vertex in a simple set (no edges). The three events are:

- **Birth:** duplicate a vertex (one organism becomes two)
- **Death:** remove a vertex
- **Immigration:** create a new vertex from nothing

```
# Use the simplest schema: just vertices (Graph with no edges needed)
# We use Graph(1) as a single-vertex pattern
println("BDI Model: vertices = organisms, no edges needed")

BDI Model: vertices = organisms, no edges needed
```

### 3.2 Define the Rules

```
# Birth: an existing organism produces an offspring
# Pattern: one vertex → Result: two vertices (original + offspring)
L_birth = Graph(1) # match one organism
I_birth = Graph(1) # preserve the parent
R_birth = Graph(2) # parent + offspring

birth_rate = 0.3
birth = ABMRule(:Birth,
    Rule(id(L_birth),
        homomorphism(I_birth, R_birth; initial=(V=[1],))),
    ContinuousHazard(Exponential(1.0 / birth_rate)))

println("Birth rule: rate = $birth_rate per organism")
Birth rule: rate = 0.3 per organism

# Death: an organism is removed
# Pattern: one vertex → Result: empty
L_death = Graph(1)
I_death = Graph(0)
R_death = Graph(0)

death_rate = 0.5
death = ABMRule(:Death,
    Rule(homomorphism(I_death, L_death), id(I_death)),
    ContinuousHazard(Exponential(1.0 / death_rate)))

println("Death rule: rate = $death_rate per organism")
Death rule: rate = 0.5 per organism

# Immigration: a new organism appears from outside
# Pattern: empty → Result: one vertex
immigration_rate = 2.0
immigration = ABMRule(:Immigration,
    Rule(id(Graph(0)),
        create(ob(terminal(Graph))))),
    ContinuousHazard(Exponential(1.0 / immigration_rate)))

println("Immigration rule: rate = $immigration_rate (population-independent)")
Immigration rule: rate = 2.0 (population-independent)
```

### 3.3 Analytical Steady State

For the BDI process with per-capita birth rate  $b$ , per-capita death rate  $d$  (where  $d > b$ ), and immigration rate  $\lambda$ , the expected steady-state population is:

$$E[N] = \frac{\lambda}{d - b}$$

```
expected_N = immigration_rate / (death_rate - birth_rate)
println("Expected steady-state population: $expected_N")
println(" (immigration=$immigration_rate / (death=$death_rate - birth=$birth_rate))")
Expected steady-state population: 10.0
```

```
(immigration=2.0 / (death=0.5 - birth=0.3))
```

### 3.4 Run the Simulation

```
abm_bdi = ABM([birth, death, immigration])

# Start with a small population
init_bdi = Graph(5)

# Track population size over time
obs_N = Observable(:N, Graph(1))

result_bdi = run!(abm_bdi, deepcopy(init_bdi);
    maxtime=50.0, maxevent=10000,
    observables=[obs_N], save_every=1.0)

println("Simulation complete: $(length(result_bdi)) events")
Simulation complete: 5407 events
```

### 3.5 Analyze Results

```
# Count events by type
event_counts = Dict{String, Int}()
for (_, _, name, _) in result_bdi.events
    event_counts[name] = get(event_counts, name, 0) + 1
end
println("Event counts:")
for (name, count) in sort(collect(event_counts))
    println("  $name: $count")
end

# Population trajectory from observables
println("\nPopulation trajectory (sampled every 1.0 time units):")
for (t, snap) in result_bdi.snapshots
    t > 20.0 && break
    println("  t=$(round(t, digits=1)): N=$(snap[:N])")
end

Event counts:
  Birth: 2832
  Death: 2482
  Immigration: 93

Population trajectory (sampled every 1.0 time units):
  t=0.0: N=5
  t=1.0: N=7
  t=2.0: N=9
  t=3.0: N=11
  t=4.0: N=13
  t=5.0: N=19
  t=6.0: N=24
  t=7.0: N=31
  t=8.0: N=36
  t=9.0: N=48
```

```

t=10.0: N=54
t=11.0: N=66
t=12.0: N=71
t=13.0: N=75
t=14.0: N=77
t=15.0: N=82
t=16.0: N=92
t=17.0: N=103
t=18.0: N=118
t=19.0: N=124
t=20.0: N=140

# Compare final population to analytical prediction
if !isempty(result_bdi.snapshots)
    # Average over second half of simulation (after burn-in)
    mid = length(result_bdi.snapshots) ÷ 2
    late_pops = [snap[:N] for (_, snap) in result_bdi.snapshots[mid:end]]
    mean_pop = sum(late_pops) / length(late_pops)
    println("Observed mean population (late simulation): $(round(mean_pop, digits=1))")
    println("Expected steady state: $expected_N")
    println("Relative error: $(round(abs(mean_pop - expected_N) / expected_N * 100, digits=1))%")
end

Observed mean population (late simulation): 282.0
Expected steady state: 10.0
Relative error: 2720.4%

```

## 4 Example 2: Predator-Prey on a Graph

This model implements Lotka-Volterra-style predator-prey dynamics on a spatial graph. Prey (rabbits) and predators (foxes) occupy vertices of a shared graph; predation requires spatial proximity (a connecting edge).

### 4.1 Predator-Prey: Birth, Death, and Predation

We model a simple predator-prey system using Graph vertices. Each species has birth, death, and interaction rules.

```

# Prey birth: a new vertex appears (prey reproduce)
prey_birth = ABMRule(:PreyBirth,
    Rule(id(Graph()), create(ob(terminal(Graph)))),
    ContinuousHazard(0.8))

# Prey death: a vertex is removed
prey_death = ABMRule(:PreyDeath,
    Rule(homomorphism(Graph(0), Graph(1)), id(Graph(0))),
    ContinuousHazard(0.1))

# Predation: an edge is removed along with its target vertex
L_pred = @acset Graph begin V=2; E=1; src=[1]; tgt=[2] end
I_pred = Graph(1)
predation = ABMRule(:Predation,
    Rule(homomorphism(I_pred, L_pred; initial=(V=[1])), id(I_pred)),
    ContinuousHazard(0.3))

println("Rules: prey birth (0.8), prey death (0.1), predation (0.3 per edge)")

```

Rules: prey birth (0.8), prey death (0.1), predation (0.3 per edge)

```
# Initial state: 10 vertices with some edges
init_pp = @acset Graph begin
    V=10; E=8
    src=[1,2,3,4,5,6,7,8]; tgt=[3,4,5,6,7,8,9,10]
end

obs_v = Observable(:Vertices, Graph(1))
abm_pp = ABM([prey_birth, prey_death, predation])
result_pp = run!(abm_pp, deepcopy(init_pp);
    maxtime=8.0, maxevent=100,
    observables=[obs_v], save_every=1.0)

println("Predator-prey: $(length(result_pp)) events")
Predator-prey: 9 events
```

## 4.2 Analyze Population Dynamics

```
pp_counts = Dict{String, Int}()
for (_, _, name, _) in result_pp.events
    pp_counts[name] = get(pp_counts, name, 0) + 1
end
println("Event counts:")
for (name, count) in sort(collect(pp_counts))
    println("  $name: $count")
end

Event counts:
  Predation: 2
  PreyBirth: 7

println("\nPopulation over time:")
for (t, snap) in result_pp.snapshots
    println("  t=$(round(t, digits=1)): vertices=$(snap[:Vertices])")
end
```

```
Population over time:
t=0.0: vertices=10
t=1.0: vertices=10
t=2.0: vertices=11
t=3.0: vertices=11
t=4.0: vertices=12
t=5.0: vertices=12
t=6.0: vertices=14
t=7.0: vertices=15
t=8.0: vertices=15
t=9.0: vertices=15
t=10.0: vertices=15
t=11.0: vertices=15
```

## 4.3 Multiple Replicates

```
println("Final vertex counts across 5 replicates:")
for rep in 1:5
```

```

Random.seed!(42 + rep)
res = run!(abm_pp, deepcopy(init_pp); maxtime=8.0, maxevent=100)
if !isempty(res.hist)
    nv = nparts(codom(right(last(res.hist))), :V)
    println(" Rep $rep: $nv vertices after $(length(res)) events")
else
    println(" Rep $rep: no events, $(nparts(init_pp, :V)) vertices")
end
end
Random.seed!(42)

```

Final vertex counts across 5 replicates:

```

Rep 1: 18 vertices after 12 events
Rep 2: 25 vertices after 19 events
Rep 3: 14 vertices after 12 events
Rep 4: 12 vertices after 8 events
Rep 5: 21 vertices after 11 events

```

TaskLocalRNG()

## 5 Example 3: Metapopulation Dynamics

In metapopulation ecology, populations live in discrete **patches** connected by **migration corridors**. Local populations undergo birth and death, while organisms migrate between connected patches.

### 5.1 Schema: Patches with Populations

We represent patches as vertices in a graph, with edges representing migration corridors. Each patch has a population of organisms modeled as a count.

```

# We use a simple graph: patches = vertices, corridors = edges
# Organisms within patches are tracked as additional vertices
# connected to their patch

```

```

@present SchMeta(FreeSchema) begin
    Patch::Ob; Corridor::Ob; Organism::Ob; Lives::Ob
    src::Hom(Corridor, Patch)
    tgt::Hom(Corridor, Patch)
    patch::Hom(Lives, Patch)
    organism::Hom(Lives, Organism)
end
@acset_type MetaPop(SchMeta)

```

```

println("Metapopulation schema:")
println(" Patch - habitat patches")
println(" Corridor - migration links between patches")
println(" Organism - individual organisms")
println(" Lives - relationship linking organisms to patches")

```

Metapopulation schema:

```

Patch - habitat patches
Corridor - migration links between patches
Organism - individual organisms
Lives - relationship linking organisms to patches

```

## 5.2 Build the Initial Landscape

Create a linear chain of patches (1-2-3-4-5) with organisms distributed across them.

```
n_patches = 5
init_organisms = [10, 5, 8, 3, 12] # initial organisms per patch

init_meta = MetaPop()

# Add patches
add_parts!(init_meta, :Patch, n_patches)

# Connect patches in a chain: 1 2 3 4 5
for i in 1:(n_patches-1)
    add_part!(init_meta, :Corridor; src=i, tgt=i+1)
    add_part!(init_meta, :Corridor; src=i+1, tgt=i)
end

# Add organisms to patches
for p in 1:n_patches
    for _ in 1:init_organisms[p]
        o = add_part!(init_meta, :Organism)
        add_part!(init_meta, :Lives; patch=p, organism=o)
    end
end

println("Landscape: $n_patches patches in a chain")
for p in 1:n_patches
    n_org = count(==(p), init_meta[:patch])
    neighbors = length(incident(init_meta, p, :src))
    println(" Patch $p: $n_org organisms, $neighbors outgoing corridors")
end

Landscape: 5 patches in a chain
Patch 1: 10 organisms, 1 outgoing corridors
Patch 2: 5 organisms, 2 outgoing corridors
Patch 3: 8 organisms, 2 outgoing corridors
Patch 4: 3 organisms, 2 outgoing corridors
Patch 5: 12 organisms, 1 outgoing corridors
```

## 5.3 Define Metapopulation Rules

### 5.3.1 Local Birth

An organism in a patch produces offspring in the same patch.

```
# Birth: match an organism-in-patch, create another organism in the same patch
L_local_birth = @acset MetaPop begin
    Patch=1; Organism=1; Lives=1; patch=[1]; organism=[1]
end

I_local_birth = @acset MetaPop begin
    Patch=1; Organism=1; Lives=1; patch=[1]; organism=[1]
end

R_local_birth = @acset MetaPop begin
    Patch=1; Organism=2; Lives=2; patch=[1, 1]; organism=[1, 2]
end
```

```

local_birth = ABMRule(:LocalBirth,
    Rule(id(L_local_birth),
        homomorphism(I_local_birth, R_local_birth;
            initial=(Patch=[1], Organism=[1], Lives=[1])),
        ContinuousHazard(Exponential(1.0 / 0.3)))

println("Local birth: rate 0.3 per organism")

Local birth: rate 0.3 per organism

```

### 5.3.2 Local Death

An organism dies and is removed from its patch.

```

# Death: match an organism-in-patch, remove the organism and the Lives link
L_local_death = @acset MetaPop begin
    Patch=1; Organism=1; Lives=1; patch=[1]; organism=[1]
end
I_local_death = @acset MetaPop begin Patch=1 end
R_local_death = @acset MetaPop begin Patch=1 end

local_death = ABMRule(:LocalDeath,
    Rule(homomorphism(I_local_death, L_local_death; initial=(Patch=[1])),
        id(I_local_death)),
    ContinuousHazard(Exponential(1.0 / 0.4)))

println("Local death: rate 0.4 per organism")

Local death: rate 0.4 per organism

```

### 5.3.3 Migration

An organism moves from one patch to an adjacent patch along a corridor.

```

# Migration: match organism-in-patch and corridor from that patch
# Result: organism moves to the target patch
L_migrate = @acset MetaPop begin
    Patch=2; Corridor=1; Organism=1; Lives=1
    src=[1]; tgt=[2]; patch=[1]; organism=[1]
end
I_migrate = @acset MetaPop begin
    Patch=2; Corridor=1; Organism=1
    src=[1]; tgt=[2]
end
R_migrate = @acset MetaPop begin
    Patch=2; Corridor=1; Organism=1; Lives=1
    src=[1]; tgt=[2]; patch=[2]; organism=[1]
end

migration = ABMRule(:Migration,
    Rule(homomorphism(I_migrate, L_migrate;
        initial=(Patch=[1,2], Corridor=[1], Organism=[1])),
        homomorphism(I_migrate, R_migrate;
            initial=(Patch=[1,2], Corridor=[1], Organism=[1])),
    ContinuousHazard(Exponential(1.0 / 0.1)))

```



```
println("Migration: rate 0.1 per organism-corridor pair")
```

Migration: rate 0.1 per organism-corridor pair

## 5.4 Run the Metapopulation Model

```
abm_meta = ABM([local_birth, local_death, migration])
```

```
# Observable: count organisms
obs_organisms = Observable(:Organisms,
  @acset MetaPop begin Organism=1 end)
```

```
result_meta = run!(abm_meta, deepcopy(init_meta);
  maxtime=30.0, maxevent=5000,
  observables=[obs_organisms], save_every=2.0)
```

```
println("Metapopulation simulation: $(length(result_meta)) events")
```

Metapopulation simulation: 396 events

## 5.5 Analyze Metapopulation Dynamics

```
# Event breakdown
meta_counts = Dict{String, Int}()
for (_, _, name, _) in result_meta.events
  meta_counts[name] = get(meta_counts, name, 0) + 1
end
println("Event counts:")
for (name, count) in sort(collect(meta_counts))
  println("  $name: $count")
end
```

Event counts:  
LocalBirth: 144  
LocalDeath: 178  
Migration: 74

```
# Total organism trajectory
println("\nTotal organisms over time:")
for (t, snap) in result_meta.snapshots
  println("  t=$(round(t, digits=1)): $(snap[:Organisms]) organisms")
end
```

Total organisms over time:  
t=0.0: 38 organisms  
t=2.0: 32 organisms  
t=4.0: 28 organisms  
t=6.0: 25 organisms  
t=8.0: 17 organisms  
t=10.0: 20 organisms  
t=12.0: 15 organisms  
t=14.0: 12 organisms  
t=16.0: 15 organisms  
t=18.0: 8 organisms  
t=20.0: 6 organisms

```

t=22.0: 7 organisms
t=24.0: 4 organisms
t=26.0: 4 organisms
t=28.0: 6 organisms
t=30.0: 4 organisms

```

## 5.6 Effect of Network Topology

Let's compare the linear chain to a fully connected landscape to see how connectivity affects population persistence.

```

function make_landscape(n_patches, init_orgs, connectivity)
    state = MetaPop()
    add_parts!(state, :Patch, n_patches)

    if connectivity == :chain
        for i in 1:(n_patches-1)
            add_part!(state, :Corridor; src=i, tgt=i+1)
            add_part!(state, :Corridor; src=i+1, tgt=i)
        end
    elseif connectivity == :complete
        for i in 1:n_patches, j in 1:n_patches
            i != j && add_part!(state, :Corridor; src=i, tgt=j)
        end
    end

    for p in 1:n_patches
        for _ in 1:init_orgs[p]
            o = add_part!(state, :Organism)
            add_part!(state, :Lives; patch=p, organism=o)
        end
    end
    return state
end

for topology in [:chain, :complete]
    init_orgs = [10, 5, 8, 3, 12]
    landscape = make_landscape(5, init_orgs, topology)
    n_corridors = nparts(landscape, :Corridor)

    res = run!(abm_meta, deepcopy(landscape);
        maxtime=20.0, maxevent=3000,
        observables=[obs_organisms], save_every=5.0)

    final_count = isempty(res.snapshots) ? sum(init_orgs) : last(res.snapshots)[2][:Organisms]
    println("$topology network ($n_corridors corridors): final organisms = $final_count after $(length(res.snapshots)) events")
end

chain network (8 corridors): final organisms = 6 after 318 events
complete network (20 corridors): final organisms = 4 after 365 events

```

## 5.7 Patch-Level Analysis

We can examine the final distribution of organisms across patches.

```

# Run a fresh simulation and examine final state
init_analysis = make_landscape(5, [10, 5, 8, 3, 12], :chain)
res_analysis = run!(abm_meta, deepcopy(init_analysis);
    maxtime=20.0, maxevent=3000, record_history=true)

if !isempty(res_analysis.hist)
    final_state = codom(right(last(res_analysis.hist)))
    println("Final patch populations:")
    for p in 1:nparts(final_state, :Patch)
        n_org = count(==(p), final_state[:patch])
        println(" Patch $p: $n_org organisms")
    end
    println("Total: $(nparts(final_state, :Organism)) organisms")
else
    println("No events occurred")
end

Final patch populations:
Patch 1: 5 organisms
Patch 2: 1 organisms
Patch 3: 0 organisms
Patch 4: 0 organisms
Patch 5: 0 organisms
Total: 6 organisms

```

## 6 Key Ecological Modeling Patterns

| Pattern              | AlgebraicABMs Technique         | Example                                       |
|----------------------|---------------------------------|-----------------------------------------------|
| Birth                | Vertex creation via Rule        | $\text{Graph}(1) \rightarrow \text{Graph}(2)$ |
| Death                | Vertex deletion via Rule        | $\text{Graph}(1) \rightarrow \text{Graph}(0)$ |
| Immigration          | Creation from empty             | $\text{Graph}(0) \rightarrow \text{Graph}(1)$ |
| Type-dependent rates | <code>ClosureState</code> timer | Rate = 0 if wrong type                        |
| Spatial interaction  | Edge in pattern                 | Predation requires edge                       |
| Migration            | Edge-dependent rewrite          | Move organism along edge                      |
| Population tracking  | <code>Observable</code>         | Count homomorphisms                           |
| Habitat structure    | Graph topology                  | Chain vs. complete network                    |

## 7 Summary

This vignette demonstrated how AlgebraicABMs naturally models ecological systems:

1. **Birth-Death-Immigration:** The simplest model validates against known analytical results ( $E[N] = \lambda/(d - b)$ ). Three rules with constant rates produce realistic stochastic population dynamics.
2. **Predator-Prey:** Typed vertices allow multiple species on a shared graph. State-dependent timers (`ClosureState`) enable type-specific rates, and edges enforce spatial predation requirements.
3. **Metapopulation:** Richer schemas (Patch, Organism, Lives, Corridor) capture multi-level ecological structure. Migration rules move organisms between connected patches, and network topology influences population persistence.

The algebraic rewriting framework ensures that all events are compositional and mathematically well-defined, while competing clocks provide exact stochastic simulation.